

Introducció a GitLab i control de versions amb Git

CFGS Desenvolupament d'Aplicacions Multiplataforma

Mòdul 0483: Implantació de sistemes operatius

Curs: 2025-2026

Objectius **3**
Conceptes clau **3**
Bloc 1: Configuració inicial **3**
Bloc 2: Crear un repositori local **4**
Bloc 3: Fer el primer commit **5**
Bloc 4: Connectar amb GitLab **5**
Bloc 5: Treballant amb canvis **7**
Bloc 5: Treballant amb branques **8**
Bloc 6: Fusionar branques **8**
Bloc 7: Crear un Mege Request **11**

Objectius

La següent pràctica pretén que l'alumne entengui el concepte de control de versions i la seva importància, creant i configurar un compte de GitLab, adquirint el domini de les comandes bàsiques de Git (clone, add, commit, push, pull), creant i gestionant repositoris i explorant els recursos principals de GitHub.

Conceptes clau

- **Git:** Sistema de control de versions distribuït (software local)
- **GitLab:** Plataforma web per allotjar repositoris Git (el "núvol")
- **Repositori (repo):** Carpeta amb historial de canvis
- **Commit:** "Fotografia" d'un moment concret del projecte
- **Branch:** Línia de desenvolupament independent
- **Remote:** Repositori en un servidor remot (GitLab)

Bloc 1: Configuració inicial

1. En primer lloc, comprovarem si el Git està instal·lat a la nostra màquina amb la comanda git --version. Si no està instal·lat (Debian/Ubuntu) executarem la línia d'ordres:
    a. sudo apt update
    b. sudo apt install git -y

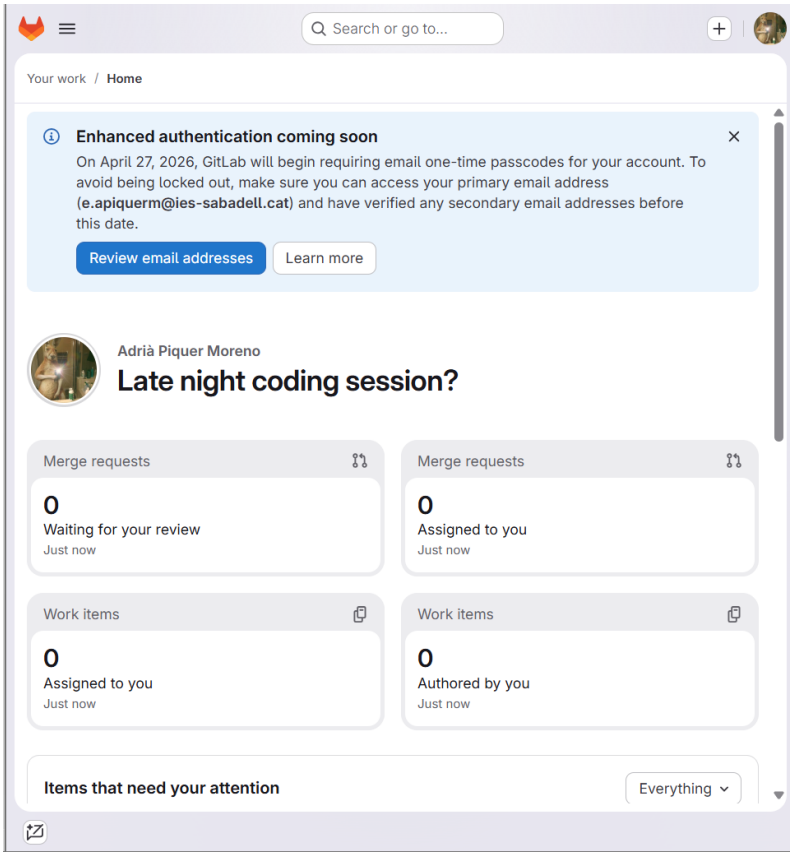
```
adria@AdriaPc:~$ git --version
bash: git: orden no encontrada
adria@AdriaPc:~$ sudo apt update
[sudo] contraseña para adria:
Obj:1 http://security.debian.org/debian-security bookworm-security InRelease
Obj:2 http://deb.debian.org/debian bookworm InRelease
Obj:3 http://deb.debian.org/debian bookworm-updates InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 410 paquetes. Ejecute «apt list --upgradable» para verlos.
adria@AdriaPc:~$ sudo apt install git -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  git-man liberror-perl patch
Paquetes sugeridos:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn ed diffutils-doc
Se instalarán los siguientes paquetes NUEVOS:
  git git-man liberror-perl patch
0 actualizados, 4 nuevos se instalarán, 0 para eliminar y 410 no actualizados.
Se necesita descargar 9.474 kB de archivos.
```

```
adria@AdriaPc:~$ git --version
git version 2.39.5
adria@AdriaPc:~$
```

2. Un cop el Git està instal·lat, ja podem començar amb la configuració bàsica.
    a. En primer lloc, hem de configurar el nom d'usuari amb la línia d'ordres:
        i. git config --global user.name "El Teu Nom"
    b. Configurarem el correu electrònic:
        i. git config --global user.email "teu.email@exemple.com"
    c. I comprovarem la configuració:
        i. git config --list

```
adria@AdriaPc:~$ git config --global user.name "Adria"
adria@AdriaPc:~$ git config --global user.email "e.apiquerm@ies-sabadell.cat"
adria@AdriaPc:~$ git config --list
user.name=Adria
user.email=e.apiquerm@ies-sabadell.cat
adria@AdriaPc:~$
```

3. Ara s'ha de crear el compte de GitLab
    a. Accedeix a https://gitlab.com i registra't amb el correu de l'Institut.
    b. Verifica el correu electrònic i configura el perfil bàsic.



Bloc 2: Crear un repositori local

4. Un cop el Git ja està instal·lat i funcional, s'ha configurat el nom i correu electrònic i podem iniciar sessió a GitLab sense problemes, ja podem crear el nostre primer repositori local.
    a. Crearem una carpeta per al nostre projecte anomenada projecteDAM i accedirem a dins.

```
adria@AdriaPc:~$ mkdir projecteDAM
adria@AdriaPc:~$ ls
Descargas   Escritorio  Música      projecteDAM  Vídeos
Documentos  Imágenes    Plantillas  Público
adria@AdriaPc:~$
```

    b. Un cop dins de la (nova) carpeta, inicialitzarem el repositori Git:
        i. git init

```
adria@AdriaPc:~$ cd projecteDAM/
adria@AdriaPc:~/projecteDAM$ git init
ayuda: Usando 'master' como el nombre de la rama inicial. Este nombre de rama pr
edeterminado
ayuda: está sujeto a cambios. Para configurar el nombre de la rama inicial para
usar en todos
ayuda: de sus nuevos repositorios, reprimiendo esta advertencia, llama a:
ayuda:
ayuda:  git config --global init.defaultBranch <nombre>
ayuda:
ayuda: Los nombres comúnmente elegidos en lugar de 'master' son 'main', 'trunk'
y
ayuda: 'development'. Se puede cambiar el nombre de la rama recién creada median
te este comando:
ayuda:
ayuda:  git branch -m <nombre>
Inicializado repositorio Git vacío en /home/adria/projecteDAM/.git/
adria@AdriaPc:~/projecteDAM$
```

    c. i comprovarem l'estat:
        i. git status

```
adria@AdriaPc:~/projecteDAM$ git status
En la rama master

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles se
guimiento)
adria@AdriaPc:~/projecteDAM$
```

d. Executa ls -la. Quin directori nou ha aparegut? Com es diu?

```
adria@AdriaPc:~/projecteDAM$ ls -la
total 12
drwxr-xr-x  3 adria adria 4096 abr 16 20:09 .
drwx------ 16 adria adria 4096 abr 16 20:08 ..
drwxr-xr-x  7 adria adria 4096 abr 16 20:09 .git
adria@AdriaPc:~/projecteDAM$
```

e. Què et diu la comanda git status en aquest moment?

```
adria@AdriaPc:~/projecteDAM$ git status
En la rama master

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles se
guimiento)
adria@AdriaPc:~/projecteDAM$
```

5. Ja podem crear contingut, en aquest cas, en forma d'arxius de text.

a. Crea un arxiu README amb echo "# El meu projecte de DAM - <EL_TEUNOM>"

```
adria@AdriaPc:~/projecteDAM$ echo "# El meu projecte de DAM - Adrià Piquer" > RE
ADME.md
adria@AdriaPc:~/projecteDAM$
```

b. I, a continuació, crearem un arxiu amb nano Main.java, afegiu un codi java bàsic.



Bloc 3: Fer el primer commit

6. Ara ja podem fer el primer commit, per veure, així, l'estat dels fitxers, mitjançant git status

    a. Quin color tenen els fitxers nous quan executes git status?

```
adria@AdriaPc:~/projecteDAM$ git status
En la rama master

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
        Main.java
        README.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa
"git add" para hacerles seguimiento)
adria@AdriaPc:~/projecteDAM$
```

    b. Què vol dir el canvi de color?

Git detecta els fitxers però no estan preparats per fer commit

    c. Què et suggereix que facis Git?

Suggereix que fem un git add

7. Ara ja podem afegir els arxius a l'staging area mitjançant la comanda git add. Tant podem fer-ho de cada arxiu, un per un, o bé, tots ells d'un sol cop amb git add .

    a. Després de fer git add, de quin color són ara els fitxers?

```
adria@AdriaPc:~/projecteDAM$ git add .
adria@AdriaPc:~/projecteDAM$ git status
En la rama master

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)
        nuevos archivos: Main.java
        nuevos archivos: README.md

adria@AdriaPc:~/projecteDAM$
```

    b. Què vol dir que estiguin "staged for commit"?

Vol dir que estan preparats per guardar-se al proxim commit.

c. Podries desfer aquesta operació? Com?

Es pot desfer amb git restore --staged

8. Arribats a aquest punt, s'ha de fer un commit, que vindria a ser com una fotografia del teu projecte en un moment determinat. Si més endavant fas canvis que no funcionen, sempre pots tornar a aquest punt.
9. Fes un commit mitjançant git commit -m "Primer commit: afegits README i Main.java". A continuació, comprova si hi ha algun canvi a l'historial amb git log.

```
adria@AdriaPc:~/projecteDAM$ git commit -m "Primer commit: afegits README i MAIN
.java"
[master (commit-raíz) f9ac926] Primer commit: afegits README i MAIN.java
 2 files changed, 6 insertions(+)
 create mode 100644 Main.java
 create mode 100644 README.md
adria@AdriaPc:~/projecteDAM$
```

a. Què apareix quan fas git log? Descriu tota la informació que veus.

```
adria@AdriaPc:~/projecteDAM$ git log
commit f9ac92679757b7a3a081a645fefcef1e9c7334bf (HEAD -> master)
Author: Adria <e.apiquerm@ies-sabadell.cat>
Date:   Thu Apr 16 20:28:38 2026 +0200

    Primer commit: afegits README i MAIN.java
adria@AdriaPc:~/projecteDAM$
```

b. Anota el número del commit (hash). Quants caràcters té?

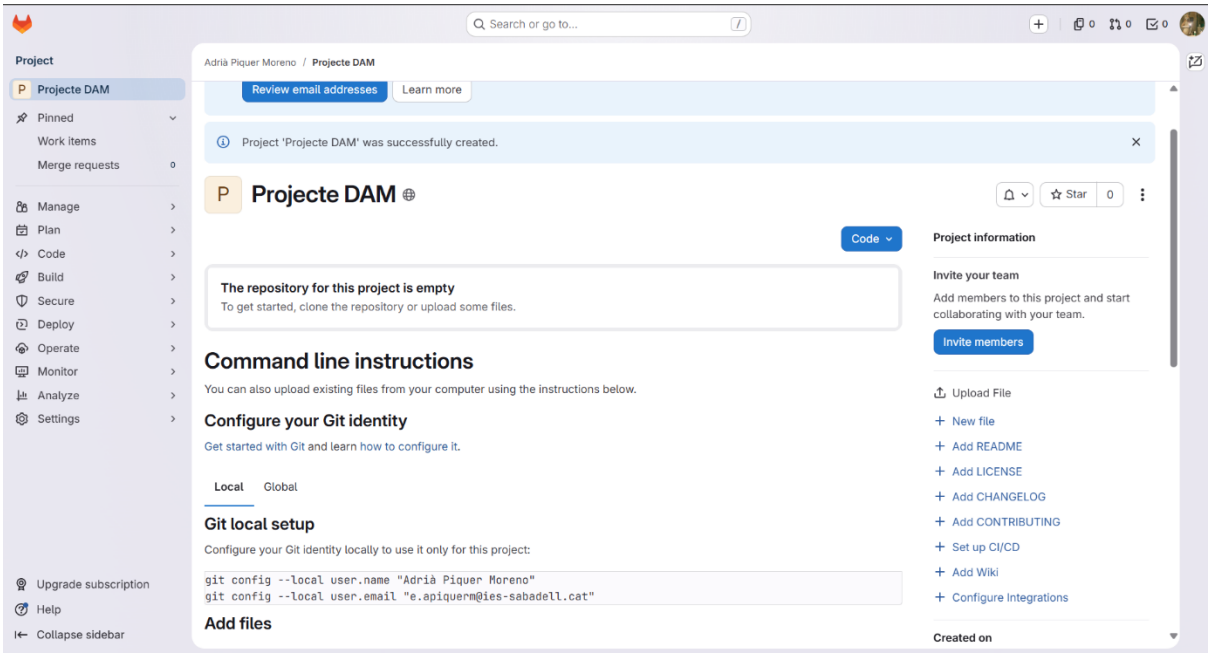
f9ac92679757b7a3a081a645fefcef1e9c7334bf, te 40 caracters

c. Per què creus que cada commit té un identificador únic?

Perquè cada commit ha de ser únic i identificable

Bloc 4: Connectar amb GitLab

10. Adreça't a GitLab.com, clica a "New project/ repository" (botó +) i crea'n un (blank project) anomenat **projecte-DAM (i descripció:** "Projecte de DAM proves Git - 1r DAM"). Posa'l **públic, Desmarca** "Initialize repository witha README" (ja el tenim local) i clica "Create project".



a. Quina diferència hi ha entre un repositori públic i un de privat?

- Que el public el pot veure tothom mentres que el privat nomes el poden veure usuaris autoritzats

b. Per què NO s'hauria de marcar "Initialize with README"? Què passaria si marquéssim aquesta opció?

- Perque ja tenim un README en local, i donaria conflicte al fer el push

11. Per connectar el repositori local amb GitLab, haurem d'Afegir el remote (copiar la URL del teu repositori de GitLab) mitjançant git remote add origin https://gitlab.com/el-teu-usuari/projecte-DAM.git

    a. Comprovar el remote amb git remote -v

```
adria@AdriaPc:~/projecteDAM$ git remote add origin https://gitlab.com/adria_piqu
er/projecte-DAM.git
adria@AdriaPc:~/projecteDAM$ git remote -v
origin  https://gitlab.com/adria_piquer/projecte-DAM.git (fetch)
origin  https://gitlab.com/adria_piquer/projecte-DAM.git (push)
adria@AdriaPc:~/projecteDAM$
```

12. Què ha aparegut quan has executat git remote -v?

    - Apareix la URL del repositori de gitlab

    a. Què vol dir el paràmetre "origin"?

       - Es el nom del repositori remot

    b. Per què apareix dues vegades (fetch i push)?

       - Perque un es per descarregar, el fetch, i el push per pujar

13. Pujar els canvis a GitLab mitjançant git branch -M main i git push -u origin main

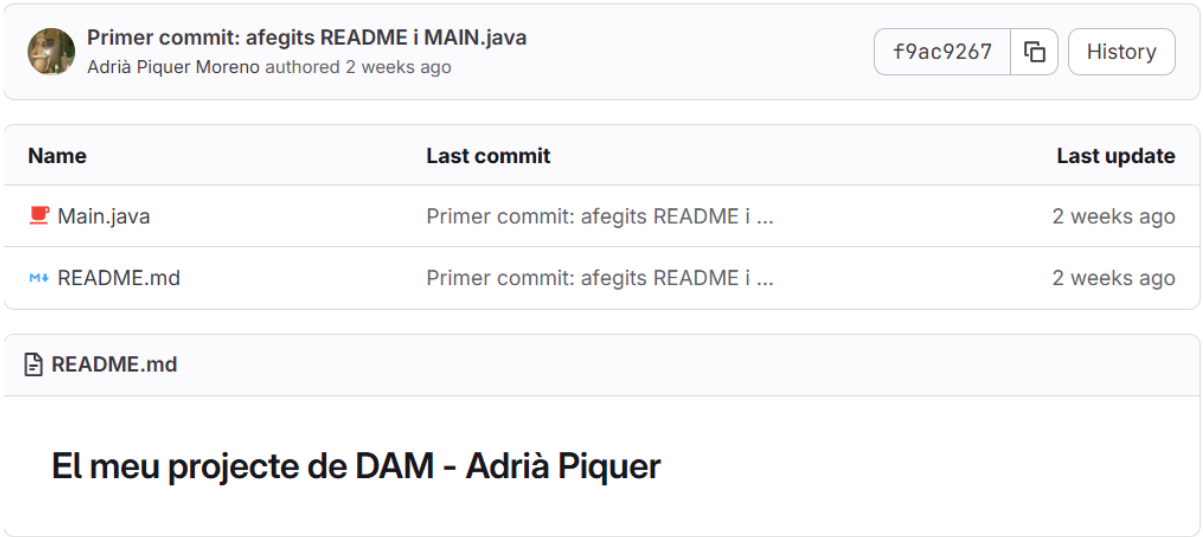
Important: Si et demana usuari i contrasenya, introdueix el teu nom d'usuari de GitLab i la contrasenya.

```
adria@AdriaPc:~/projecteDAM$ git branch -M main
adria@AdriaPc:~/projecteDAM$ git push -u origin main
Username for 'https://gitlab.com': adria_piquer
Password for 'https://adria_piquer@gitlab.com':
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (4/4), 420 bytes | 420.00 KiB/s, listo.
Total 4 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://gitlab.com/adria_piquer/projecte-DAM.git
 * [new branch]      main -> main
rama 'main' configurada para rastrear 'origin/main'.
adria@AdriaPc:~/projecteDAM$
```

14. Per verificar que els canvis i les actualitzacions s'han fet efectives, t'has d'adreçar a GitLab.com, a la pàgina principal del repositori i comprovar quins arxius s'han pujat.



a. El README.md es mostra formatat a la part inferior?



15. Veus el missatge de l'últim commit al costat de cada fitxer?

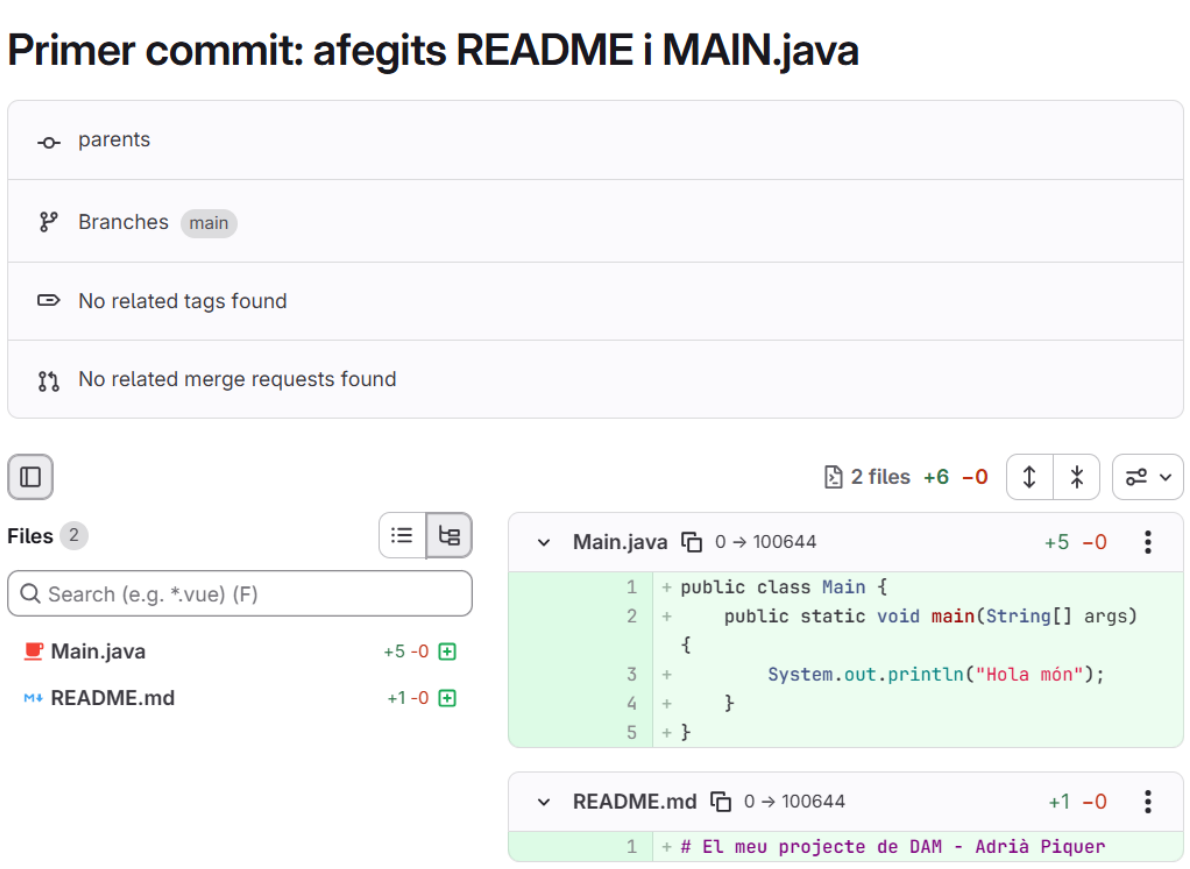


16. Fes clic sobre el fitxer README.md i confirma que es mostra el contingut complet.

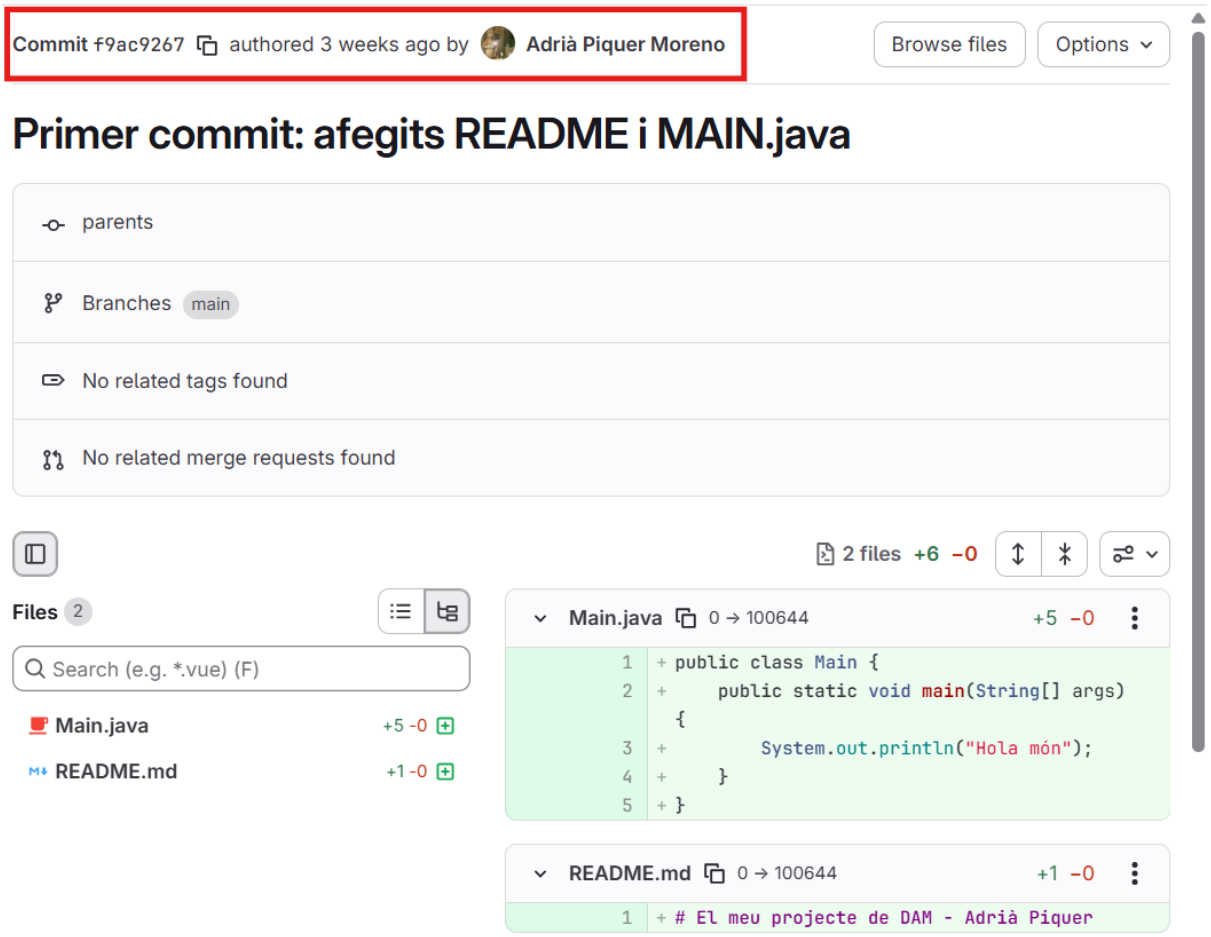
    a. Hi ha un botó per veure el text sense format?



    b. Com puc fer per veure els canvis que s'hi poden haver fet?

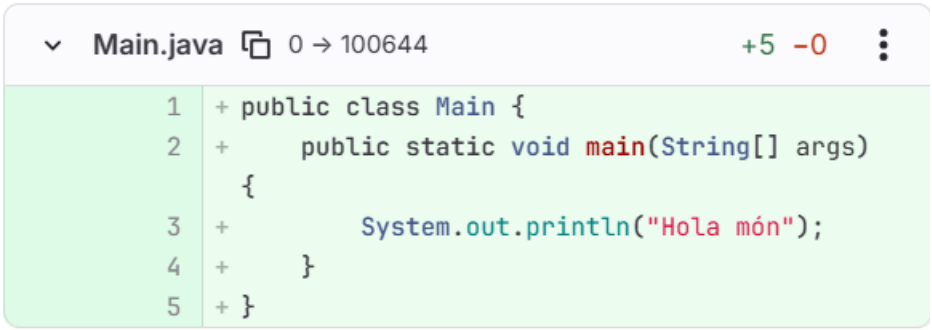


c. On has d'anar per veure els commits que has fet? Apareix la data i hora exacta? Apareix el teu nom d'usuari?



d. Fes clic sobre el teu commit (sobre el missatge). Què apareix en verd? I en vermell? Per què?

- Apareix en verd les lines que he afegit i apareix en vermell les lines que he eliminat, com que nomes he afegit nomes surt verd:





e. Què passa si cliques sobre una de les línies. Pots fer alguna cosa?

- Si cliquem a sobre d'una linea podem afegir comentaris a aquella linea

f. Torna a la pàgina principal. Fes clic sobre Main.java. Pots llegir el contingut? Hi ha un botó "Edit"? Què et permet fer?

- permet modificar el fitxer des del navegador sense necessitat d'utilitzar un editor local

Bloc 5: Treballant amb canvis

17. Com que ja tenim pujats diferents documents al nostre GitLab, ja podem començar a fer-hi canvis, per exemple, modificant el README.md.

a. Des de l'intèrpret d'ordres, Afegeix al README el següent contingut:

```
## Descripció
Aquest repositori conté un projecte de Java per fer proves amb Gitlab

## Contingut
Projecte bàsic en Java

##Autor [el teu nom]
Curs: 1r DAM
Data: [data actual]
```

b. Comprova els canvis abans de fer commit amb git status i git diff README.md

```
adria@AdriaPc:~/projecteDAM$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de
 trabajo)
        modificados:     README.md

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
adria@AdriaPc:~/projecteDAM$
```

```
adria@AdriaPc:~/projecteDAM$ git diff README.md
diff --git a/README.md b/README.md
index f369ff6..81603a7 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,11 @@
 # El meu projecte de DAM - Adrià Piquer
+
+## Descripció
+Aquest repositori conté un projecte de Java per fer proves amb Gitlab
+
+## Contingut
+Projecte bàsic en Java
+
+##Autor [Adrià Piquer Moreno]
+Curs: 1r DAM
+Data: 08/05/2026
adria@AdriaPc:~/projecteDAM$
```

c. Què et mostra la comanda git diff?

- Ens mostra les diferències entre versions

d. De quin color són les línies que has afegit?

- Les linies que he afegit surten de color verd

e. Si haguessis esborrat línies, de quin color serien?

- Si hagues esborrat linies sortirien de color vermell

f. Per què creus que és útil veure les diferències abans de fer commit?

- Per revisar errors abans del commit.

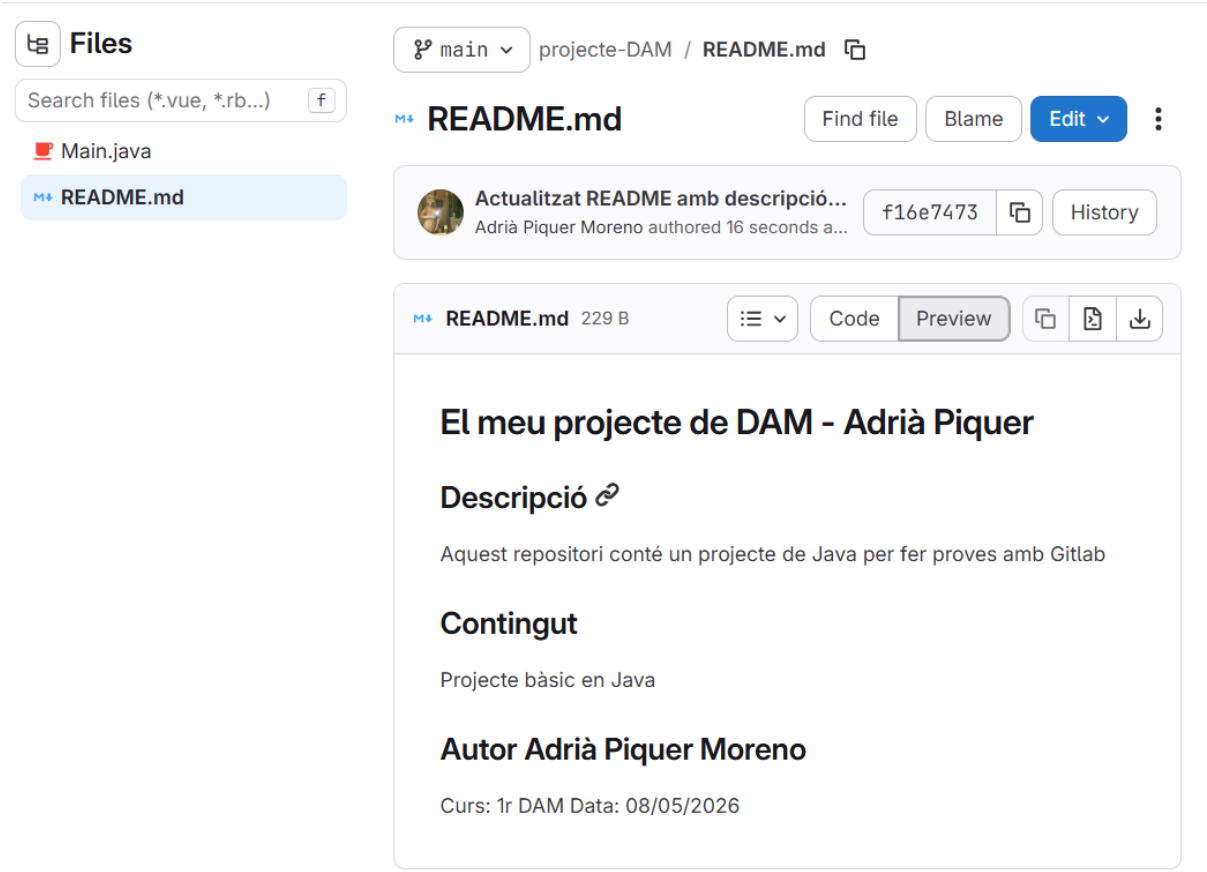
18. Ara ja es pot afegir i fer el commit corresponent amb git add README.md i git commit -m "Actualitzat README amb descripció del projecte"

```
adria@AdriaPc:~/projecteDAM$ git add README.md
adria@AdriaPc:~/projecteDAM$ git commit -m "Actualitzat README amb descripció de
l projecte"
[main 896f5c6] Actualitzat README amb descripció del projecte
 1 file changed, 10 insertions(+)
adria@AdriaPc:~/projecteDAM$
```

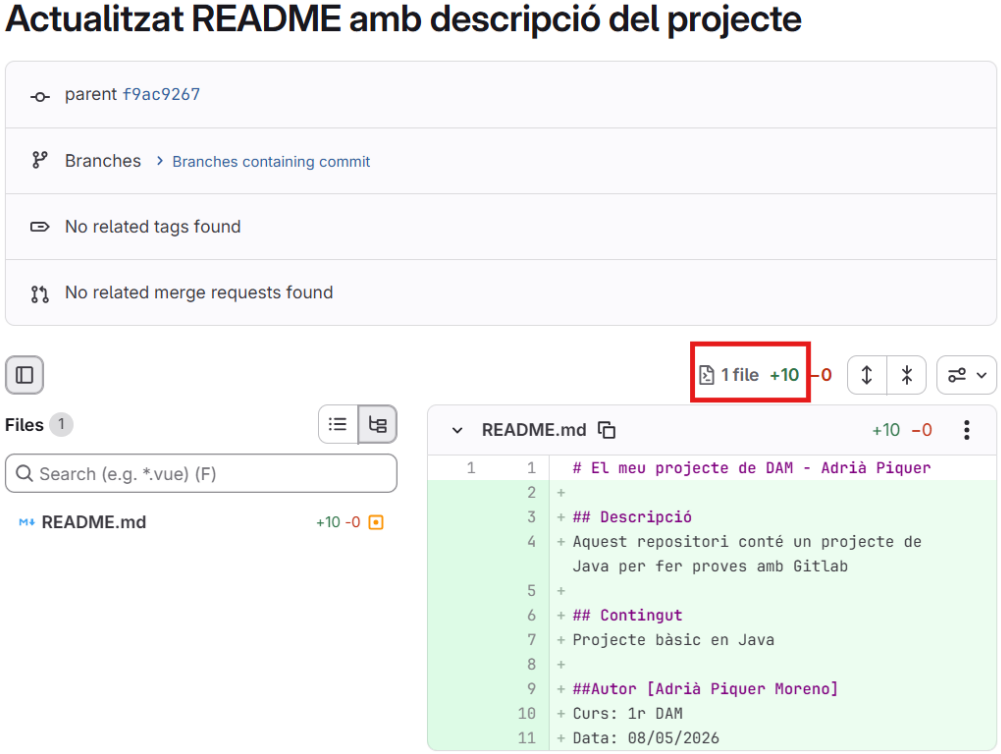
a. Puja-ho tot al GitLab amb git push origin main

```
adria@AdriaPc:~/projecteDAM$ git push origin main
Username for 'https://gitlab.com': adria_piquer
Password for 'https://adria_piquer@gitlab.com':
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 484 bytes | 484.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://gitlab.com/adria_piquer/projecte-DAM.git
   f9ac926..896f5c6  main -> main
adria@AdriaPc:~/projecteDAM$
```

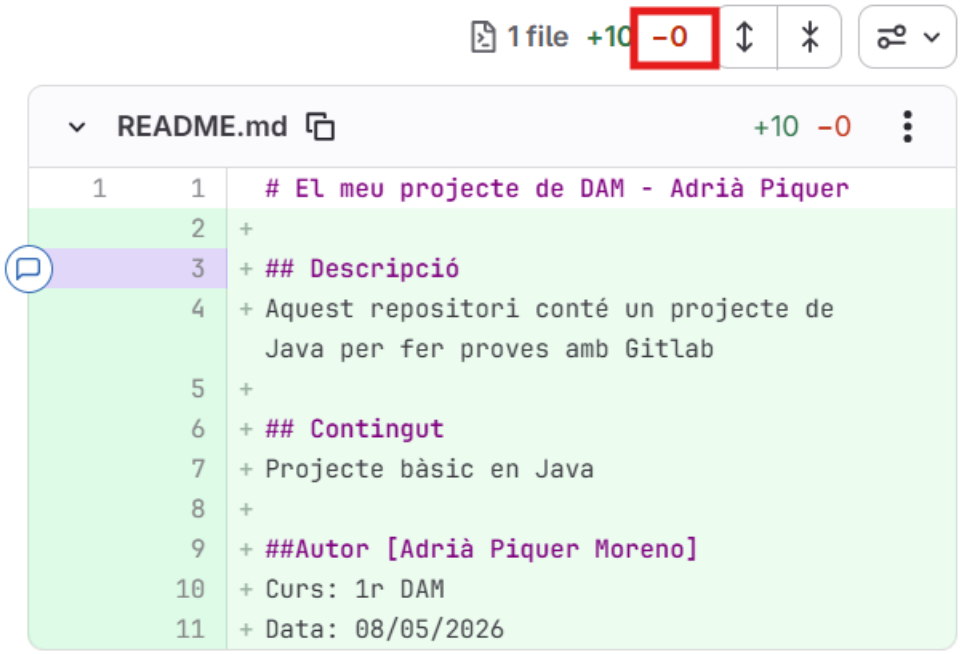
19. Comprova els canvis a GitLab anant a la pàgina del teu repositori a GitHub. Actualitza la pàgina (F5 o Ctrl+R) i mira el README.md.



a. Quantes línies has afegit al README.md? (número verd)



b. Quantes línies has esborrat? (número vermell)



c. GitLab mostra el fitxer complet o només els canvis?

- Mostra el fitxer complet

d. Pots veure qui va fer cada canvi i quan?



Bloc 5: Treballant amb branques

Una branca és com una línea temporal alternativa del teu projecte. Pots experimentar amb nous canvis sense afectar la versió principal (main). Si els canvis funcionen bé, els pots fusionar; si no, simplement esborres la branca.

20. Per veure les branques existents, s'ha d'executar git branch.

    a. Quantes branques tens ara?

```
adria@AdriaPc:~/projecteDAM$ git branch
* main
adria@AdriaPc:~/projecteDAM$
```

    b. Com saps en quina branca estàs actualment?

Ho podem saber per *, aixo indica on estem

    c. Què indica l'asterisc (*) o el color diferent?

Indica la branca en la que estas

21. Per crear una branca nova, tenim, dues opcions: git checkout -b codi-nou o bé, fer-ho en dues passes amb git branch codi-nous i git checkout apunts-nous

```
adria@AdriaPc:~/projecteDAM$ git checkout -b codi-nou
Cambiado a nueva rama 'codi-nou'
adria@AdriaPc:~/projecteDAM$
```

22. Per verificar el canvi de branca, has d'executar git branch.

```
adria@AdriaPc:~/projecteDAM$ git branch
* codi-nou
  main
adria@AdriaPc:~/projecteDAM$
```

    a. En quina branca estàs ara?

Ara estem a la branca codi-nou que acabem de crear.

    b. Què passarà amb els canvis que facis en aquesta branca? Afectaran la branca main?

No, els canvis no afectaran a la branca main a no ser que fem un merge

23. Crear un fitxer nou a la nova branca anomenat codi_nou.java amb el contingut que et sembli i fes commit i push d'aquest fitxer

```
  GNU nano 7.2                        codi_nou.java *
public class codi_nou {
    public static void main(String[] args) {
        System.out.println("Nova branca");
    }
}
```

```
adria@AdriaPc:~/projecteDAM$ git add codi_nou.java
adria@AdriaPc:~/projecteDAM$ git commit -m "Afegir codi_nou.java"
En la rama codi-nou
nada para hacer commit, el árbol de trabajo está limpio
adria@AdriaPc:~/projecteDAM$ git push origin codi-nou
Username for 'https://gitlab.com': adria_piquer
Password for 'https://adria_piquer@gitlab.com':
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 416 bytes | 416.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
remote:
remote: To create a merge request for codi-nou, visit:
remote:   https://gitlab.com/adria_piquer/projecte-DAM/-/merge_requests/new?merg
e_request%5Bsource_branch%5D=codi-nou
remote:
To https://gitlab.com/adria_piquer/projecte-DAM.git
 * [new branch]      codi-nou -> codi-nou
adria@AdriaPc:~/projecteDAM$
```

a. Executa ls. Quants fitxers veus?

```
adria@AdriaPc:~/projecteDAM$ ls
codi_nou.java  Main.java  README.md
adria@AdriaPc:~/projecteDAM$
```

b. Ara canvia a la branca main: git checkout main

```
adria@AdriaPc:~/projecteDAM$ git checkout main
Cambiado a rama 'main'
Tu rama está actualizada con 'origin/main'.
adria@AdriaPc:~/projecteDAM$
```

c. Torna a executar ls. Quants fitxers veus ara? On està el fitxer que acabes de crear? Què demostra això sobre les branques?

```
adria@AdriaPc:~/projecteDAM$ ls
Main.java  README.md
adria@AdriaPc:~/projecteDAM$
```

El fitxer que acabem de crear esta a la branca codi-nou, demostra que cada branca és independent.

d. Executa git log i compta quants commits tens a cada branca.

```
adria@AdriaPc:~/projecteDAM$ git log --oneline
f16e747 (HEAD -> main, origin/main) Actualitzat README amb descripció del projec
te
117ae9b Actualitzat README amb descripció del projecte
896f5c6 Actualitzat README amb descripció del projecte
f9ac926 Primer commit: afegits README i MAIN.java
adria@AdriaPc:~/projecteDAM$
```

Bloc 6: Fusionar branques

24. Torna a la branca principal amb `git checkout main` i fusiona la branca codi-nou amb main (`git merge codi-nou`).

```
adria@AdriaPc:~/projecteDAM$ git checkout main
Ya en 'main'
Tu rama está actualizada con 'origin/main'.
adria@AdriaPc:~/projecteDAM$
```

a. Després del merge, executa ls. Què veus ara?

```
adria@AdriaPc:~/projecteDAM$ git merge codi-nou
Actualizando f16e747..a1582eb
Fast-forward
 codi_nou.java | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 codi_nou.java
adria@AdriaPc:~/projecteDAM$
```

```
adria@AdriaPc:~/projecteDAM$ ls
codi_nou.java  Main.java  README.md
adria@AdriaPc:~/projecteDAM$
```

b. Executa git log. Pots veure els commits de la branca codi-nou? Què ha passat exactament amb el merge?

```
adria@AdriaPc:~/projecteDAM$ git log --oneline
a1582eb (HEAD -> main, origin/codi-nou, codi-nou) Afegir codi_nou.java
f16e747 (origin/main) Actualitzat README amb descripció del projecte
117ae9b Actualitzat README amb descripció del projecte
896f5c6 Actualitzat README amb descripció del projecte
f9ac926 Primer commit: afegits README i MAIN.java
adria@AdriaPc:~/projecteDAM$
```

Sí, ara tambe surten els commits de la branca codi-nou.

El merge ha fusionat l'historial i els canvis de les branques dins del main.

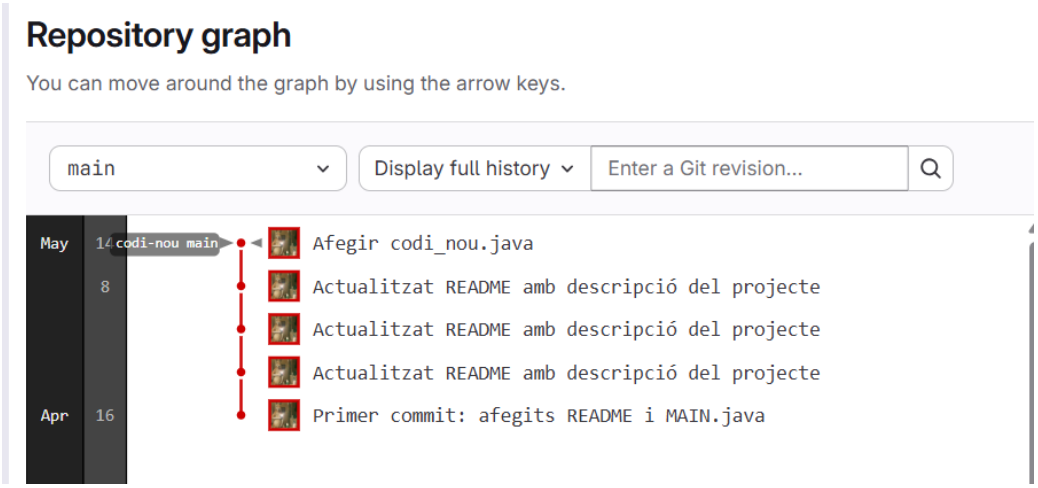
c. La branca codi-nou encara existeix? Comprova-ho amb git branch

```
adria@AdriaPc:~/projecteDAM$ git branch
  codi-nou
* main
adria@AdriaPc:~/projecteDAM$
```

d. Pujar els canvis fusionats a GitLab amb git push origin main

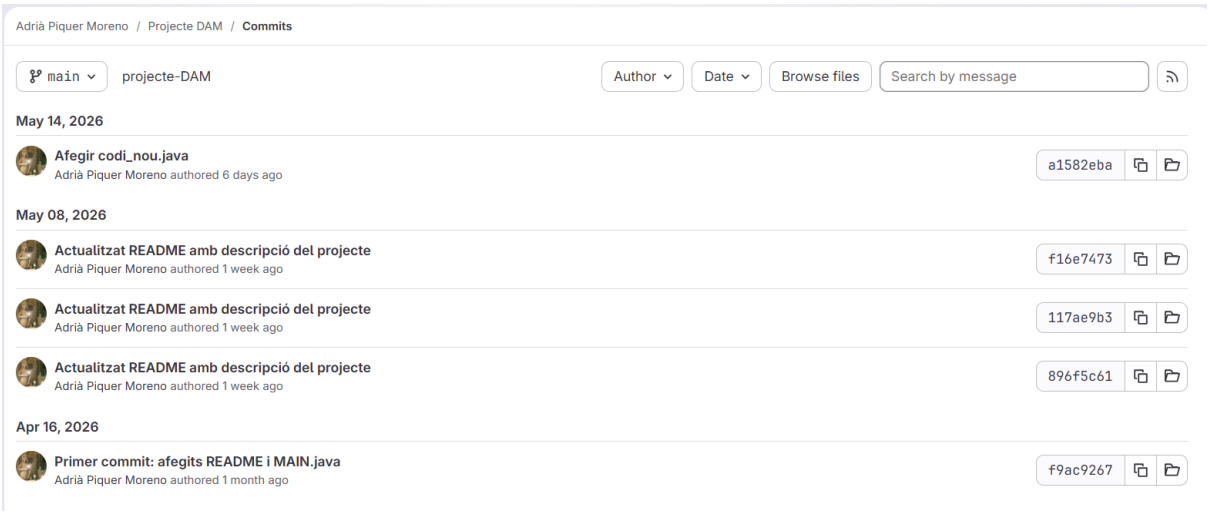
```
adria@AdriaPc:~/projecteDAM$ git push origin main
Username for 'https://gitlab.com': adria_piquer
Password for 'https://adria_piquer@gitlab.com':
Total 0 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://gitlab.com/adria_piquer/projecte-DAM.git
   f16e747..a1582eb  main -> main
adria@AdriaPc:~/projecteDAM$
```

25. Actualitza la pàgina a la branca main i comprova quins arxius veus a main després del merge.
    a. Fes clic al desplegable "Code" i comprova si hi ha un gràfic de les branques? Apareix com es fusionen les branques amb main. Què representen les línies?



Les lineas representan l'historial de commits i la relació entre las diferents branques.

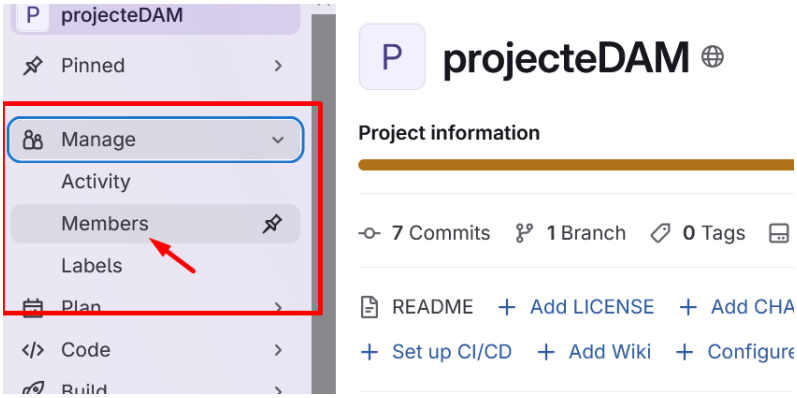
26. Torna a la pàgina principal i comprova els commits:
    a. Veus el commit del merge? Té algun missatge especial o icona diferent?
    b. Pots identificar quan es van crear i fusionar les branques
    c. Opcionalment, esborra la branca ja fusionada amb git branch -d codi-nou

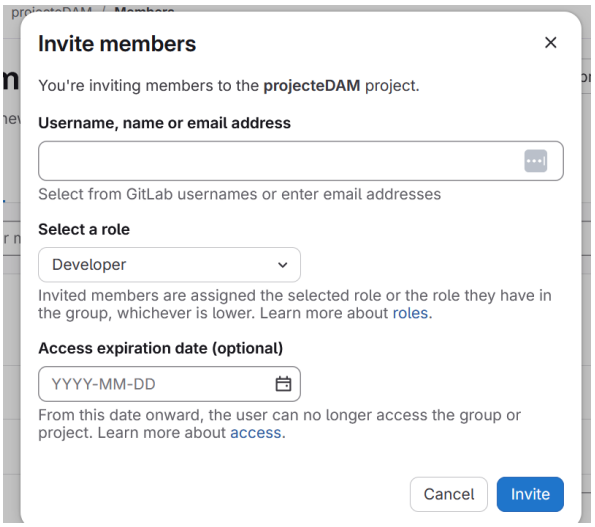


No ha aparegut el commit

A partir d'aquest punt, per tal d'explotar els recursos de compartició del GitLab, haurem de treballar col·laborant amb algun/a company/a de classe. Tot allò que hagi de fer el/la company/a, apareixerà en un color diferent.

27. Adreça't a GitLab i, estan dins del nostre projecte ves a l'apartat Manage → Members i convida al teu company de classe perquè treballi en el projecte. Dona-li el rol de Developer





a. Què vol dir afegir algú com a developer? Quins permisos tindrà sobre el teu repositori? Podria esborrar el teu repositori?

Agregar a algu com Developer significa que podrá fer push, crear branques, crear merge requests , modificar arxius, pero no podrá borrar el repositori ni cambiar configuracions del projecte.

28. Demana al col·laborador/a que cloni el teu repositori amb git clone https://gitlab.com/teu-usuari/projectedam.git i que, a continuació, executi un cd projecteDAM
29. Col·laborador/a, executa ls i comprova si veus tots els arxius del/de la teva company/a. A continuació, executa git log i comprova si pots veure tot l'historial de commits.
    a. Què diferencia git clone de simplement descarregar els arxius?
30. Crea una nova branca per als canvis amb git checkout -b contribucions-alumneB i crea el següent arxiu (de contribucions) anomenat CONTRIBUCIONS.md.

```
# Contribucions

## Alumne B - [El teu nom]
### Revisions realitzades
He revisat el codi i blablabla

### Propostes de millora
    - proposta1
    - proposta2

### Data de la contribució
[Data actual]
```

    a. Fes un commit per l'arxiu amb git add CONTRIBUCIONS.md i indica l'actualització amb git commit -m "Afegides contribucions de l'alumne B - [El teu nom]" i puja'l amb git push origin contribucions-alumneB

Bloc 7: Crear un Merge Request

31. [Alumne B a GitLab] Ves al projecte fes F5 i veuràs un missatge que et suggereix si vols fer un Merge Request.

a. Després de fer push, GitLab mostra un missatge que has fet un push de la nova branca "contribucions-alumneB" i trobaràs un botó "Create merge request". Fes-hi click.
    i. Títol: "Contribucions de [nom Alumne B]"
    ii. Descripció: Explica què has afegit o modificat
    iii. Veuràs que hi ha marcat el Delete source branch, deixa'l

32. Abans de crear el MR, desplaça't cap avall i comprova si pots veure tots els commits de la teva branca i els canvis, pots?
    a. Fes clic a "Create mergerequest"

33. A la pàgina principal del teu repositori de GitLab veuràs un número al costat de Code →"Merge requests" (hauria de ser 1). Fes clic a "Merge requests".
    a. Comprova si pots veure el MR de l'alumne B i clica-la
    b. Confirma que hi ha una etiqueta "open" verda.
34. Dins del Merge Request, llegeix la descripció que ha escrit l'Alumne B i fes clic a la pestanya "Changes".
    a. Confirma que pots veure tots els canvis en verd (afegits) i vermell (esborrats).
    b. Si tot està bé, torna a "Overview" i desplaça't fins a baix, veuràs que està marcat que s'esborri la branca després de fer el merge. Fes clic en el botó "Merge" (i confirma-ho).
    c. Després del merge, comprova que l'etiqueta canvia a "Merged" (blau) i que s'ha esborrat la branca.
35. Després de fusionar el MR, torna a la pàgina principal del projecte
    a. Ves a "Code" → "Repository graph". Què hi pots veure?
36. Per últim, baixa els canvis acceptats al repositori local amb git checkout main i git pull origin main